Five-Database Graph Benchmark

A Reproducible Performance Evaluation of CognoDB, Neo4j Aura, Memgraph, FalkorDB, and ArangoDB

Project	Graph Database Performance Benchmark
Databases	CognoDB, Neo4j Aura, Memgraph, FalkorDB, ArangoDB
Dataset	Pokec-derived graph dataset
Dataset size	169,870 nodes and 100,000 relationships
Latency benchmark	100 measured runs with warm-up runs
Mixed workload	80% reads / 20% writes
Concurrency	1, 10, and 40 clients/workers
ArangoDB deployment	Local Docker container
Report	Benchmark methodology, results, analysis and reproducibility

Purpose of this report: This document provides a complete technical description of the graph database benchmarking project, including database setup, dataset preparation, benchmark implementation, workload design, measurement methodology, results, analysis, fairness considerations, limitations, and reproducibility instructions.

1. Executive Summary

This project evaluates five graph database systems using a common Pokec-derived graph dataset and a scripted benchmark framework. The evaluated systems are CognoDB, Neo4j Aura, Memgraph, FalkorDB, and ArangoDB.

The benchmark measures both individual query latency and concurrent mixed read/write throughput. The latency suite contains node-count aggregation, indexed point lookup, and 1-hop, 2-hop, and 3-hop graph traversal workloads. The mixed workload contains 80% reads and 20% writes and is evaluated at concurrency levels of 1, 10, and 40.

For the four initially compared systems, FalkorDB recorded the lowest measured latency across the five latency workloads and the highest measured mixed-workload throughput. ArangoDB was subsequently added as the fifth database and successfully loaded with the same 169,870-node and 100,000-edge dataset. Its benchmark produced independent latency and concurrency measurements.

The results are benchmark-specific. They should not be interpreted as universal rankings because database architecture, cloud tier, deployment topology, hardware, network conditions, indexing, query language, configuration, and client overhead can affect measured performance.

2. Project Objectives

- Build a reusable graph database benchmarking framework.
- Prepare one common graph dataset for all evaluated platforms.
- Implement database-specific connectors for reliable connectivity.
- Load the graph dataset into each benchmark platform.
- Measure aggregation and lookup latency.
- Measure 1-hop, 2-hop, and 3-hop graph traversal latency.
- Measure mixed read/write throughput.
- Evaluate concurrency at 1, 10, and 40 workers.
- Use warm-up and repeated runs to reduce transient effects.
- Save raw benchmark results as CSV files.
- Generate comparison tables and charts.
- Calculate relative performance and speedup.
- Document methodology, fairness, limitations and reproducibility.
- Prepare a GitHub-ready project and final technical report.

3. Databases Evaluated

Database	Role in Project	Benchmark Status
CognoDB	Primary comparison platform	Benchmarked
Neo4j Aura	Managed graph database comparison	Benchmarked
Memgraph	Graph database comparison	Benchmarked
FalkorDB	Graph database comparison	Benchmarked
ArangoDB	Fifth database added for assignment completeness	Loaded and benchmarked

Important: ArangoDB was added after the original four-database comparison. It is therefore documented separately in the benchmark results where the earlier four-database result files do not yet contain ArangoDB.

4. Dataset Preparation

The benchmark uses a processed Pokec-derived graph dataset. The final benchmark input is stored as a CSV file containing source and target identifiers for graph relationships.

Property	Value
Dataset	Pokec-derived processed dataset
Input file	data/processed/pokec_100k_edges.csv
Columns	source, target
Relationships	100,000
Unique nodes	169,870
Graph type	Directed relationship dataset

The dataset was explicitly verified before loading into ArangoDB. The validation confirmed that the CSV contained exactly 100,000 rows and the expected source/target columns.

5. Data Loading and Database Setup

A database-specific loader was implemented for ArangoDB. The loader connects using environment variables rather than storing credentials in source code. The graph is represented using a vertex collection named **nodes** and an edge collection named **relationships**.

The loader also creates an ArangoDB graph named **pokec_graph**. During development, a collection-deletion error occurred because the relationships collection was still part of the graph. The cleanup logic was corrected to delete the graph before deleting its collections.

After the correction, the loader successfully performed the complete cycle of cleaning old data, creating collections, inserting nodes, inserting relationships, creating the graph, and verifying the final counts.

ArangoDB Component	Implemented Configuration
Host	http://localhost:8529
Database	_system
Graph	pokec_graph
Node collection	nodes
Edge collection	relationships
Nodes loaded	169,870
Relationships loaded	100,000
Deployment	Docker

6. Database Connectivity and Credential Handling

Database credentials are not embedded directly in benchmark source code. The project uses environment variables loaded through **python-dotenv**. This approach allows the same benchmark code to be reused with different credentials and prevents passwords and connection details from being committed to the repository.

The ArangoDB connector was independently tested before the dataset loading stage. The connection test successfully reported the ArangoDB server version and confirmed a working database connection.

Repository security rule: passwords, API keys, private connection URIs, and other secrets must not be committed to GitHub. A .env.example file can be provided containing variable names and placeholder values.

7. Benchmark Methodology

The benchmark was designed around the principle that every database should receive the same logical workload and dataset. Query syntax is database-specific, but the intended operation is kept equivalent across systems.

Methodology Item	Configuration
Dataset	Same Pokec-derived graph dataset
Latency warm-up	Warm-up executions before measurement
Measured latency runs	100 runs per workload
Latency metrics	Average, minimum, maximum, median and P95
Mixed workload	80% reads / 20% writes
Mixed workload duration	Approximately 30 seconds
Concurrency levels	1, 10, 40
Output	CSV result files
Visualization	PNG comparison charts

8. Benchmark Workloads

Workload	Purpose
Node Count	Counts the number of nodes in the graph. This represents an aggregation-style workload.
Relationship Count	Counts graph relationships. For ArangoDB this is measured using the relationships edge collection.
Indexed / Point Lookup	Retrieves a specific node using an identifier lookup. The exact implementation is database-specific while the
1-Hop Traversal	Traverses one relationship level from a selected starting node.
2-Hop Traversal	Traverses exactly two relationship levels from the starting node.
3-Hop Traversal	Traverses exactly three relationship levels from the starting node.
Mixed Read/Write	Runs a concurrent workload consisting of approximately 80% reads and 20% writes.

9. Benchmark Implementation

The benchmark implementation is written in Python. The framework contains connection handling, query definitions, warm-up execution, timing, statistical calculations, CSV generation, and concurrency testing.

- **Environment loading:** python-dotenv is used to read database credentials.

- **Database connectivity:** database-specific connectors establish authenticated sessions.
- **Timing:** time.perf_counter() is used for high-resolution elapsed-time measurement.
- **Warm-up:** initial executions are performed before measured runs.
- **Repeated measurements:** each latency workload is executed repeatedly.
- **Statistics:** average, minimum, maximum, median and P95 are calculated.
- **CSV output:** benchmark results are stored in the results directory.
- **Concurrency:** mixed workloads use multiple workers to simulate concurrent clients.
- **Visualization:** result CSV files are used to generate comparison charts.

10. ArangoDB Implementation

ArangoDB was implemented as the fifth graph database in the project. The database was deployed locally using Docker and exposed on port 8529. The Python benchmark uses the python-arango client library.

The implementation includes a connector, dataset loader, graph creation logic, data verification, latency benchmark, and mixed read/write workload benchmark.

The loader verified the final database contents as **169,870 nodes** and **100,000 relationships**. The ArangoDB benchmark then measured six latency workloads and three concurrency levels.

11. ArangoDB Benchmark Results

Workload	Average ms	Minimum ms	Maximum ms	Median ms	P95 ms
Node Count	64.5845	58.8156	85.6189	63.3397	72.9163
Relationship Count	54.2713	51.0057	60.7268	53.9590	58.6215
Indexed Lookup	3.0577	1.9311	5.3244	2.8945	4.5414
1-Hop	46.7648	43.2507	49.0612	47.4670	48.7620
2-Hop	46.3274	43.0116	51.8651	47.0127	48.7773
3-Hop	46.4933	43.0550	63.9874	46.7905	48.6026

The ArangoDB results show very low point-lookup latency relative to its aggregation and traversal workloads. Its indexed lookup measured approximately 3.06 ms on average. The 1-hop, 2-hop and 3-hop traversals remained close to 46–47 ms in this particular test.

Important: ArangoDB results should be compared with the other databases only after confirming that query semantics, indexing, deployment resources, warm-up configuration and client-side timing are equivalent.

12. ArangoDB Mixed Read/Write Results

Concurrency	Operations	Elapsed sec	Throughput ops/sec
1	804	30.02	26.78
10	7701	30.04	256.36
40	17814	30.06	592.68

ArangoDB throughput increased substantially as concurrency increased. The measured throughput rose from 26.78 operations per second at concurrency 1 to 256.36 operations per second at concurrency 10 and 592.68 operations per second at concurrency 40.

13. Four-Database Latency Results

Workload	FalkorDB	Neo4j Aura	Memgraph	CognoDB
Node Count	1.41 ms	91.21 ms	198.75 ms	318.71 ms
Point Lookup	1.84 ms	76.75 ms	165.71 ms	308.79 ms
1-Hop	4.56 ms	77.69 ms	174.79 ms	324.82 ms
2-Hop	7.81 ms	77.55 ms	170.81 ms	320.95 ms
3-Hop	33.52 ms	78.06 ms	178.80 ms	315.52 ms

Within the original four-database comparison, FalkorDB recorded the lowest average latency for all five workloads. Neo4j Aura consistently followed FalkorDB, while Memgraph and CognoDB recorded higher measured latency in this benchmark configuration.

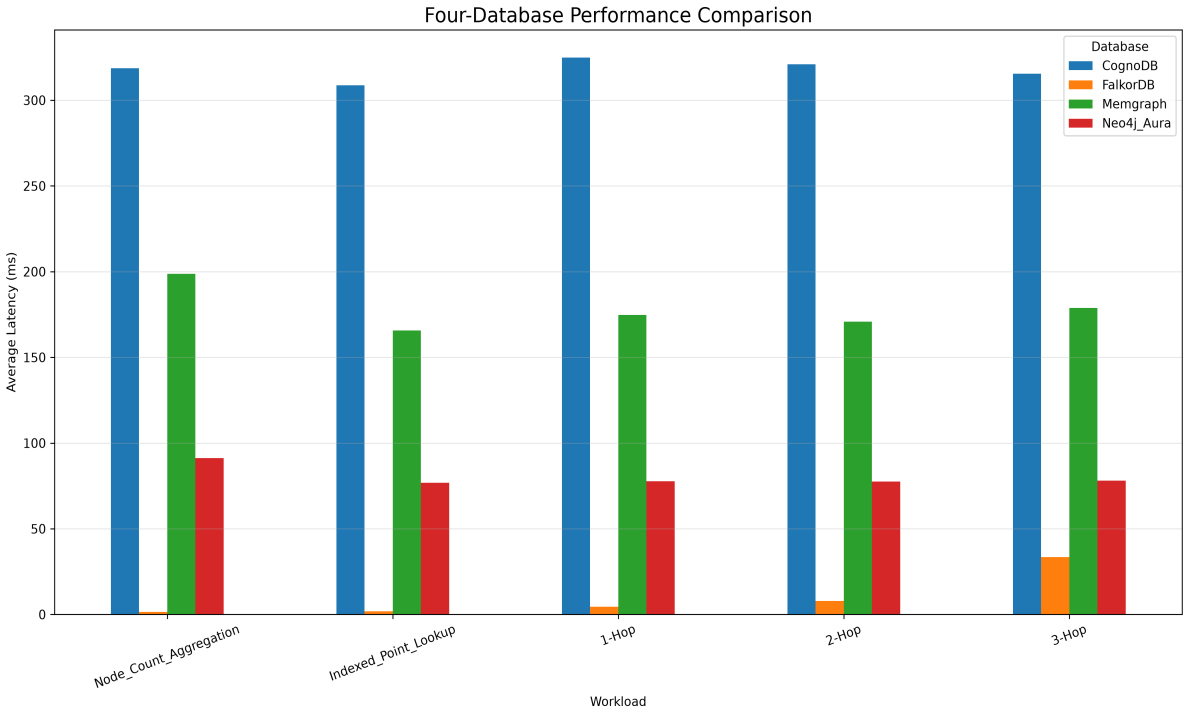

Figure 1. Graph database latency comparison.

14. Four-Database Mixed Workload Results

Concurrency	FalkorDB	Neo4j Aura	Memgraph	CognoDB
1	491.63	10.31	6.00	3.11
10	1542.55	119.48	58.08	22.13
40	1494.16	453.23	230.27	15.07

FalkorDB recorded the highest measured throughput at each tested concurrency level in the original four-database comparison. Neo4j Aura also showed a strong increase in throughput as concurrency increased, particularly between 10

and 40 workers.

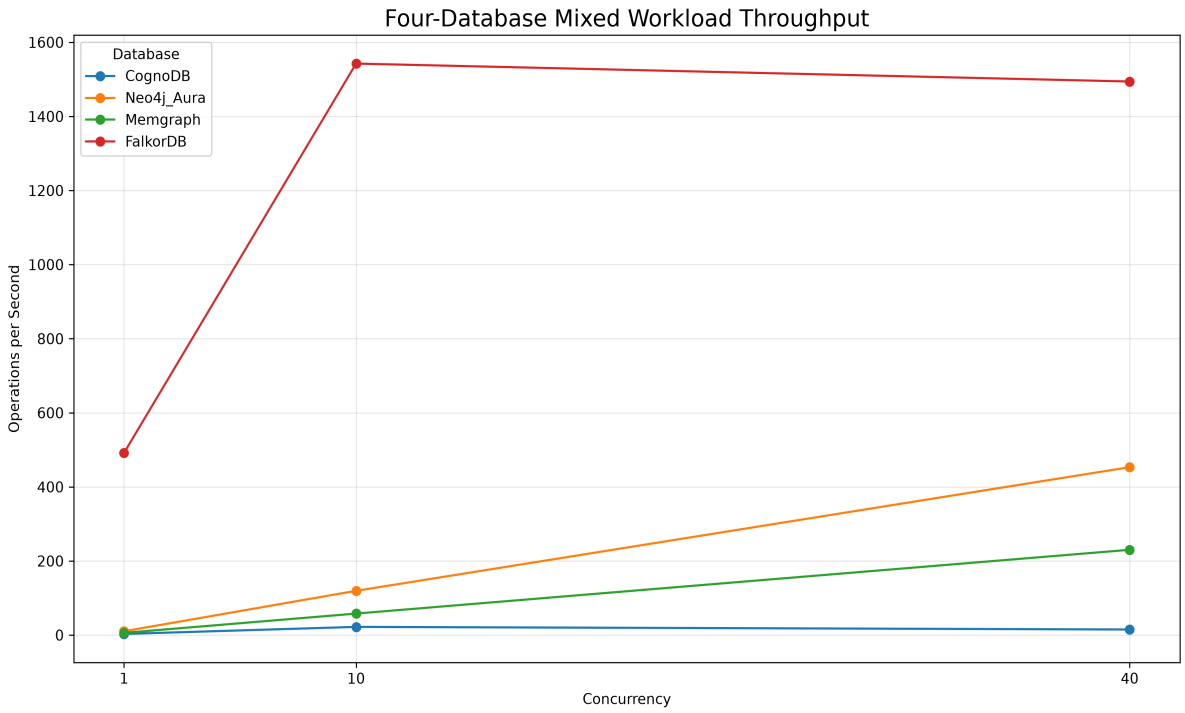

Figure 2. Mixed read/write workload throughput comparison.

15. Relative Speedup Analysis

Workload	Neo4j Aura	Memgraph	FalkorDB
Node Count	3.49×	1.60×	225.76×
Indexed Lookup	4.02×	1.86×	167.60×
1-Hop	4.18×	1.86×	71.24×
2-Hop	4.14×	1.88×	41.09×
3-Hop	4.04×	1.76×	9.41×

The speedup figures describe how much lower the measured latency was relative to CognoDB for the original four-database experiment. For example, a 225.76× figure means the measured CognoDB average latency was approximately 225.76 times the FalkorDB average for that workload.

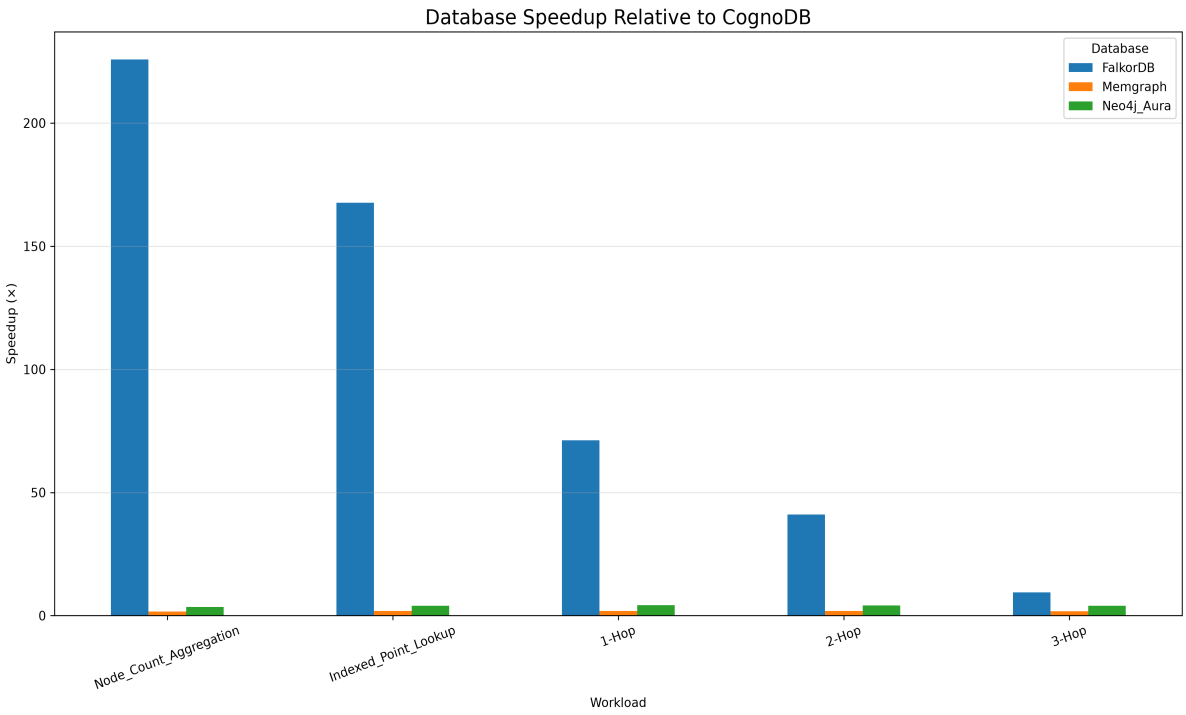

Figure 3. Relative latency speedup compared with CognoDB.

16. Analysis and Interpretation

- FalkorDB produced the lowest measured latency across all five latency workloads in the original four-database comparison.
- The largest latency advantage occurred in node-count aggregation, where FalkorDB averaged approximately 1.41 ms compared with approximately 318.71 ms for CognoDB.
- FalkorDB also showed very low point-lookup latency at approximately 1.84 ms.
- As traversal depth increased from one hop to three hops, FalkorDB latency increased, showing that deeper traversal was more expensive even for the fastest system.
- Neo4j Aura maintained a relatively stable latency profile across the traversal workloads.
- Memgraph and CognoDB showed higher latency under the tested workload and configuration.
- In the mixed workload, FalkorDB achieved the highest throughput among the original four systems at all three concurrency levels.
- ArangoDB showed strong scaling within its own benchmark: throughput increased from 26.78 ops/sec at concurrency 1 to 592.68 ops/sec at concurrency 40.
- The ArangoDB result demonstrates why the fifth database must be included in the final comparison matrix rather than leaving the project as a four-database benchmark.

Root-cause interpretation: differences between graph databases can arise from their internal storage engines, indexing structures, query planners, traversal execution strategies, caching, concurrency models, network overhead, deployment architecture, and resource allocation. The benchmark identifies measured differences but does not by itself prove a single architectural cause for every difference.

17. Methodology and Fairness

Methodology and fairness represent a major evaluation criterion because benchmark results are meaningful only when the comparison is performed consistently.

Fairness Requirement	How It Is Addressed
Same data	The benchmark is based on the same processed Pokec-derived graph dataset.
Same logical workloads	Aggregation, point lookup and traversal workloads are defined consistently.
Warm-up	Warm-up executions are performed before measured latency runs.
Repeated runs	Latency is measured over repeated executions and summarized statistically.
Concurrency sweep	Mixed workload is tested at concurrency 1, 10 and 40.
Credential isolation	Credentials are read from environment variables rather than source code.
Caveats	Deployment and resource differences are explicitly acknowledged.

Fairness caveat: a completely identical hardware and network environment may not be possible when comparing managed cloud services with locally hosted databases. The final README should therefore document each platform's deployment type, resource limits, region where relevant, and any free-tier restrictions.

18. Reproducibility and Code Quality

The project is structured so that database setup, loading, benchmarking and result generation are performed through scripts rather than manual query execution. This makes the experiment easier to repeat and audit.

- Python virtual environment is used for project dependencies.
- Database-specific connector scripts validate connectivity.
- Dataset loading is scripted.
- ArangoDB deployment is reproducible using Docker.
- Benchmark execution is scripted.
- Latency results are automatically written to CSV.
- Mixed workload results are automatically written to CSV.
- Comparison scripts generate summary files and visualizations.
- Environment variables are used for credentials.
- The README should document installation, environment variables and execution commands.
- Dependencies should be pinned in requirements.txt before final submission.

Recommended final one-command workflow: after dependencies and database credentials are configured, the repository should provide a single documented command or script that runs the required benchmark sequence and writes the results into the results/ directory.

19. Generated Results and Artifacts

Artifact	Purpose
Benchmark Python scripts	Execute latency and mixed-workload benchmarks.
Database connectors	Connect to each graph database.
Dataset loaders	Load the common graph dataset.
Latency CSV files	Store repeated-run measurements.
Mixed workload CSV files	Store concurrency throughput results.

Artifact	Purpose
Comparison CSV files	Combine database results for analysis.
Performance PNG	Visualize latency comparisons.
Speedup PNG	Visualize relative performance.
Mixed workload PNG	Visualize throughput versus concurrency.
Final PDF report	Present methodology, results and analysis.

20. Recommended GitHub Repository Structure

data/
processed/
pokec_100k_edges.csv
connectors/
arangodb.py
cognodb.py
neo4j.py
memgraph.py
falkordb.py
results/
benchmark CSV files
comparison CSV files
PNG charts
benchmarks/
database benchmark scripts
load_arangodb.py
benchmark_arangodb.py
create_comparison.py
plot_comparison_log.py
requirements.txt
.env.example
.gitignore
README.md

21. Assignment Requirement Coverage

Evaluation Criterion	Project Coverage
Methodology & fairness — 25%	Common dataset, logical workload definitions, warm-up, repeated runs, concurrency sweep and explic
Completeness of metrics — 20%	Latency and mixed read/write workloads are implemented. Final five-database matrix should include al
Reproducibility & code quality — 20%	Scripted loaders, connectors, benchmark runners, CSV outputs and environment-based credentials.
README & analysis — 15%	README should contain methodology, environment specifications, dataset details, results matrix, char
Communication — 20%	This report provides a structured explanation of the project, implementation, methodology, results, inte

Final status: the core technical benchmark implementation is substantially complete. Before submission, the remaining important work is to consolidate ArangoDB into the same five-database results matrix, verify that every required metric is available for all five platforms, pin dependencies, finalize the README, and ensure that no credentials or private connection strings are committed.

22. Limitations and Caveats

- The results depend on the exact dataset and workload definitions used.

- Managed cloud databases and locally hosted databases may not have identical hardware or network conditions.
- Free-tier resource limits can differ between platforms.
- Client-library overhead may contribute to measured latency.
- Database indexing and configuration can affect lookup performance.
- Caching and warm-state behavior can affect repeated measurements.
- A single dataset size does not establish performance for all graph sizes.
- The benchmark measures selected workloads rather than every possible graph database operation.
- Measured performance should not be interpreted as a universal ranking.
- Root-cause explanations should be supported by query plans or system documentation where possible.

23. Recommended Improvements Before Final Submission

- Generate a single five-database latency CSV containing all platforms.
- Generate a single five-database mixed-workload CSV containing all platforms.
- Regenerate the charts so that ArangoDB appears in every comparison chart.
- Add database version, deployment type, region and resource information to the README.
- Pin exact Python dependency versions in requirements.txt.
- Create .env.example without real passwords.
- Add a one-command benchmark runner.
- Add warm-versus-cold measurements if time permits.
- Repeat the complete benchmark independently to assess variance.
- Add standard deviation to the latency result matrix if required.
- Document any platform-specific query differences.
- Commit only reproducible source code and non-sensitive benchmark artifacts.

24. Conclusion

This project developed a practical and reproducible framework for evaluating graph database performance. The implementation progressed from dataset preparation and database connectivity through automated data loading, graph construction, latency benchmarking, concurrency testing, CSV result generation and visualization.

The original four-database experiment showed FalkorDB achieving the lowest measured latency and highest mixed-workload throughput among FalkorDB, Neo4j Aura, Memgraph and CognoDB under the tested configuration.

ArangoDB was subsequently implemented as the fifth database. Its dataset was successfully loaded and verified at 169,870 nodes and 100,000 relationships. The completed ArangoDB benchmark measured node count, relationship count, indexed lookup, 1-hop, 2-hop and 3-hop latency, together with mixed read/write throughput at concurrency levels 1, 10 and 40.

The most important final step is therefore not another database installation, but consolidation: all five platforms should be represented in one consistent results matrix, with the same required metrics, clearly documented resource conditions and honest caveats. Once this consolidation, dependency pinning and README work is completed, the repository will be much closer to a submission-ready benchmark package.

25. GitHub Submission Deliverables

- Benchmark source code.
- Database connector implementations.
- Dataset loading scripts.
- Benchmark workload runners.
- CSV result files.
- Comparison and analysis scripts.
- Performance charts.
- Pinned requirements.txt.
- .env.example containing variable names only.
- README with complete reproducibility instructions.
- README results matrix covering all five databases.
- Methodology and fairness documentation.
- Environment and resource specifications.
- Dataset description.
- Limitations and caveats.
- Final PDF technical report.

Repository: github.com/Hanifa246/four-database-graph-benchmark